NES-VMC 算法的核心思想正是将原系统前K个激发态的求解问题，等价地转化为一个“扩展系统”的基态求解问题。 所以我希望你可以基于 Netket 框架，实现 NES-VMC 算法。避免我自己写很多计算工具函数 完全使用 Netket 里的 API. 在原文中 $\hat{\mathcal{H}} = \hat{H}_1 \oplus \hat{H}_2 \oplus \cdots \hat{H}_K $ 而在下面的代码中 原系统的哈密顿量: `ha = nkx.operator.from_pyscf_molecule(mol)` 如果基于 Netket 框架, 怎么得到 $\hat{\mathcal{H}}$? 其中`ha` 是原系统的哈密顿量, 而`hi_ensemble` 是扩展系统的 Hilbert 空间. `ha`是 netket.experimental.operator.ParticleNumberAndSpinConservingFermioperator2nd

```
import jax
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import numpy as np
from pyscf import gto, scf, fci
from flax import nnx

#==============================================================================
# 1. 全局参数 & H₂ 分子定义
#==============================================================================
K = 2  # NES-VMC 要计算的低激发态数量（基态+1个激发态）
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
mf = scf.RHF(mol).run(verbose=0)

# FCI 精确基准（4个态）
cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()
print("="*60)
print("H₂ FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0]) * 27.2114
    print(f"E{i} = {e:.8f} Ha  |  激发能: {exc:.4f} eV")

# 转为 NetKet 哈密顿量
ha = nkx.operator.from_pyscf_molecule(mol)

# 1. 定义原始 Hilbert 空间（同之前）
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1,1),
)
# 2. 定义 NES-VMC 参数
K = 2  # 想要同时采样的组态数
# 3. 修正点：使用指数运算符 ** 来构建扩展 Hilbert 空间
hi_ensemble = hi ** K
# 4. 后续的采样器设置保持不变
edges = [(0, 1), (2, 3)] #edges 是对 hi 的 edges
```

```python
g = nk.graph.Graph(edges=edges)
# hi_ensemble.all_states()[0].shape # (12,)

hi_K = nk.hilbert.TensorHilbert(hi**K)
hi_K.all_states().shape #(4**K,4*K)

single_rule = nk.sampler.rules.FermionHopRule(hilbert=hi, graph=g)
rules_tuple = tuple(single_rule for _ in range(K))
tensor_rule = nk.sampler.rules.TensorRule(hilbert=hi_K, rules=[single_rule]*K)
```